Module-3
Name : Narendra kumar S
Date 26-07-2025

<u>Mini Project</u>

Holiday Package

The tour package details like package Id, source place, destination place, basic fare and number of days for each package should be stored in a table. Calculate the package cost based on the basic fare and the number of days for each package.

Instance_variable	Data type
package_id	String
source_place	String
destination_place	String
no_of_days	int
package_cost	double

Create a model class to represent this with attributes, getters and setters.
Calculate the package cost for each package, based on the conditions given below,

noOfDays	discount %
<=5	0% (No discount)
>5 and <=8	3%
>8 and <=10	5%
>10	7%

Package Cost = ((Basic fare x number of days)-discount) + GST
The package cost should be calculated based on the basic fare and the number of days. The discount should be calculated depending on the number of days as given in the above table and deducted from the calculated package cost. Finally, a GST of 12% of the calculated package cost got after the discount, should be added to get the final package cost.
For example: If a package has a basic fare as Rs.3000 and the number of days as 15, then the package cost will be (3000*15), which is Rs. 45000.00. Since the number of days is 15, the discount percentage will be 7%. So, the discount will be (45000.0*(7/100)) which is Rs. 3150.00. Now, 12% of GST needs to be added. So the GST will be ((45000.0-3150.0)*(12/100)) which is Rs. 5022.00.
Therefore, the total cost for this package will be (((3000*15)-3150.0)+5022.0) which is Rs. 46872.00.

Develop as a menu-driven application. This application should show options
1. Add Package details
2. Display all package details
3. Search for a package with package id
4. Calculate package cost based on package id

Use layered architecture (Client (Main class), Service Layer, Dao Layer)
Classes to be created:

Model class - Package –attributes (packageid, source, destination, no_of_days and package cost)
PackageDao interface – declare the methods for adding package details, displaying all packages, calculating package cost and searching a package from the collection.
PackageDaoImpl class – implement the method declared in PackageDao interface
PackageService interface – declare the methods – addPackage, findPackageById ,fetchAllPackages , calculatePackageCost
PackageServiceImpl class –implement the methods of PackageService and call the methods of PackageDao class.
Main class – Create object of service class and call the methods by getting choice from the user.

Validation:
The packageId should be validated before calculating the package cost; only if the packageId is valid, the Package object should be added to the list. The packageId should be in the following format.

The packageId should contain exactly 7characters.

If the packageId is valid, then calculate the package cost, else throw a user defined Exception “InvalidPackageIdException” with a message "Invalid Package Id".

Package.java

```
package com.packageapp.model;
public class Package {
  private String packageId;
  private String sourcePlace;
  private String destinationPlace;
  private int noOfDays;
  private double basicFare;
  private double packageCost;
  public Package() {}
  public Package(String packageId, String sourcePlace, String destinationPlace, int noOfDays, double
basicFare) {
      this.packageId = packageId;
      this.sourcePlace = sourcePlace;
      this.destinationPlace = destinationPlace;
      this.noOfDays = noOfDays;
      this.basicFare = basicFare;
  }

  public String getPackageId() {
                return packageId;
        }
        public void setPackageId(String packageId) {
                this.packageId = packageId;
        }
        public String getSourcePlace() {
                return sourcePlace;
        }
        public void setSourcePlace(String sourcePlace) {
```

```
            this.sourcePlace = sourcePlace;
        }
        public String getDestinationPlace() {
            return destinationPlace;
        }
        public void setDestinationPlace(String destinationPlace) {
            this.destinationPlace = destinationPlace;
        }
        public int getNoOfDays() {
            return noOfDays;
        }
        public void setNoOfDays(int noOfDays) {
            this.noOfDays = noOfDays;
        }
        public double getBasicFare() {
            return basicFare;
        }
        public void setBasicFare(double basicFare) {
            this.basicFare = basicFare;
        }
        public double getPackageCost() {
            return packageCost;
        }
        public void setPackageCost(double packageCost) {
            this.packageCost = packageCost;
        }
        @Override
    public String toString() {
        return "Package ID: " + packageId + ", Source: " + sourcePlace + ", Destination: " +
destinationPlace +
               ", Days: " + noOfDays + ", Basic Fare: " + basicFare + ", Package Cost: " + packageCost;
    }
}
```

Package Dao

```
package com.packageapp.dao;

import java.util.List;
import com.packageapp.model.Package;

public interface PackageDao {
    void addPackage(Package pkg);
    List<Package> getAllPackages();
    Package getPackageById(String id);
    void calculatePackageCost(Package pkg);
}
```

```java
package com.packageapp.dao;

import java.util.ArrayList;
import java.util.List;
import com.packageapp.model.Package;

public class PackageDaoImpl implements PackageDao {
    private List<Package> packageList = new ArrayList<>();

    @Override
    public void addPackage(Package pkg) {
        packageList.add(pkg);
    }

    @Override
    public List<Package> getAllPackages() {
        return packageList;
    }

    @Override
    public Package getPackageById(String id) {
        for (Package pkg : packageList) {
            if (pkg.getPackageId().equals(id)) {
                return pkg;
            }
        }
        return null;
    }

    @Override
    public void calculatePackageCost(Package pkg) {
        double basicCost = pkg.getBasicFare() * pkg.getNoOfDays();
        double discount = 0.0;
        int days = pkg.getNoOfDays();
        if (days > 5 && days <= 8) discount = basicCost * 0.03;
        else if (days > 8 && days <= 10) discount = basicCost * 0.05;
        else if (days > 10) discount = basicCost * 0.07;
        double discountedCost = basicCost - discount;
        double gst = discountedCost * 0.12;
        pkg.setPackageCost(discountedCost + gst);
    }
}
```

Package service

```
package com.packageapp.service;

import java.util.List;
import com.packageapp.model.Package;
import com.packageapp.exception.InvalidPackageIdException;

public interface PackageService {
    void addPackage(Package pkg) throws InvalidPackageIdException;
    List<Package> fetchAllPackages();
    Package findPackageById(String id);
    void calculateCost(String id) throws InvalidPackageIdException;
}
```

```
package com.packageapp.service;

import java.util.List;
import com.packageapp.dao.PackageDao;
import com.packageapp.dao.PackageDaoImpl;
import com.packageapp.model.Package;
import com.packageapp.exception.InvalidPackageIdException;

public class PackageServiceImpl implements PackageService {
    private PackageDao dao = new PackageDaoImpl();

    private boolean validatePackageId(String id) {
        return id != null && id.length() == 7;
    }

    @Override
    public void addPackage(Package pkg) throws InvalidPackageIdException {
        if (!validatePackageId(pkg.getPackageId())) {
            throw new InvalidPackageIdException("Invalid Package Id");
        }
        dao.addPackage(pkg);
    }

    @Override
    public List<Package> fetchAllPackages() {
        return dao.getAllPackages();
    }

    @Override
    public Package findPackageById(String id) {
        return dao.getPackageById(id);
    }
```

```
    @Override
    public void calculateCost(String id) throws InvalidPackageIdException {
        if (!validatePackageId(id)) {
            throw new InvalidPackageIdException("Invalid Package Id");
        }
        Package pkg = dao.getPackageById(id);
        if (pkg != null) {
            dao.calculatePackageCost(pkg);
        } else {
            System.out.println("Package not found.");
        }
    }
}
```

Userinterface

```
package com.packageapp;

import java.util.Scanner;
import java.util.List;
import com.packageapp.model.Package;
import com.packageapp.service.PackageService;
import com.packageapp.service.PackageServiceImpl;
import com.packageapp.exception.InvalidPackageIdException;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        PackageService service = new PackageServiceImpl();

        while (true) {
            System.out.println("\n----- Holiday Package System -----");
            System.out.println("1. Add Package Details");
            System.out.println("2. Display All Packages");
            System.out.println("3. Search Package by ID");
            System.out.println("4. Calculate Package Cost by ID");
            System.out.println("5. Exit");
            System.out.print("Enter your choice: ");
            int choice = sc.nextInt();

            try {
                switch (choice) {
                    case 1:
                        sc.nextLine();
                        System.out.print("Enter Package ID: ");
                        String id = sc.nextLine();
                        System.out.print("Enter Source Place: ");
```

```
                String source = sc.nextLine();
                System.out.print("Enter Destination Place: ");
                String dest = sc.nextLine();
                System.out.print("Enter No of Days: ");
                int days = sc.nextInt();
                System.out.print("Enter Basic Fare: ");
                double fare = sc.nextDouble();
                Package pkg = new Package(id, source, dest, days, fare);
                service.addPackage(pkg);
                System.out.println("Package Added Successfully!");
                break;

            case 2:
                List<Package> allPackages = service.fetchAllPackages();
                for (Package p : allPackages) {
                    System.out.println(p);
                }
                break;

            case 3:
                sc.nextLine();
                System.out.print("Enter Package ID to Search: ");
                String searchId = sc.nextLine();
                Package found = service.findPackageById(searchId);
                if (found != null) System.out.println(found);
                else System.out.println("Package Not Found.");
                break;

            case 4:
                sc.nextLine();
                System.out.print("Enter Package ID to Calculate Cost: ");
                String calcId = sc.nextLine();
                service.calculateCost(calcId);
                Package updated = service.findPackageById(calcId);
                if (updated != null) {
                    System.out.println("Updated Package Cost: " +
updated.getPackageCost());
                }
                break;

            case 5:
                System.out.println("Thank You for using our App");
                sc.close();
                System.exit(0);

            default:
                System.out.println("Invalid choice!");
        }
```

```
            } catch (InvalidPackageIdException e) {
                System.out.println("Error: " + e.getMessage());
            }
        }
    }
}
```

Output

```
----- Holiday Package System -----
1. Add Package Details
2. Display All Packages
3. Search Package by ID
4. Calculate Package Cost by ID
5. Exit
Enter your choice: 1
Enter Package ID: Pk00001
Enter Source Place: chennai
Enter Destination Place: kumbakonam
Enter No of Days: 2
Enter Basic Fare: 400
Package Added Successfully!
```

```
----- Holiday Package System -----
1. Add Package Details
2. Display All Packages
3. Search Package by ID
4. Calculate Package Cost by ID
5. Exit
Enter your choice: 2
Package ID: Pk00001, Source: chennai, Destination: kumbakonam, Days: 2, Basic Fare: 400.0, Package Cost: 0.0
```

```
----- Holiday Package System -----
1. Add Package Details
2. Display All Packages
3. Search Package by ID
4. Calculate Package Cost by ID
5. Exit
Enter your choice: 3
Enter Package ID to Search: Pk00001
Package ID: Pk00001, Source: chennai, Destination: kumbakonam, Days: 2, Basic Fare: 400.0, Package Cost: 0.0

----- Holiday Package System -----
```

```
----- Holiday Package System -----
1. Add Package Details
2. Display All Packages
3. Search Package by ID
4. Calculate Package Cost by ID
5. Exit
Enter your choice: 4
Enter Package ID to Calculate Cost: Pk00001
Updated Package Cost: 896.0
```

```
----- Holiday Package System -----
1. Add Package Details
2. Display All Packages
3. Search Package by ID
4. Calculate Package Cost by ID
5. Exit
Enter your choice: 5
Thank You for using our App
```